



DesignWare MSHC CTest

Software Sample

dwc_mshc_ctest_sample

Copyright Notice and Proprietary Information

© 2020 Synopsys, Inc. All rights reserved. This Synopsys software and all associated documentation are proprietary to Synopsys, Inc. and may only be used pursuant to the terms and conditions of a written license agreement with Synopsys, Inc. All other use, reproduction, modification, or distribution of the Synopsys software or the associated documentation is strictly prohibited.

Destination Control Statement

All technical data contained in this publication is subject to the export control laws of the United States of America. Disclosure to nationals of other countries contrary to United States law is prohibited. It is the reader's responsibility to determine the applicable regulations and to comply with them.

Disclaimer

SYNOPSYS, INC., AND ITS LICENSORS MAKE NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, WITH REGARD TO THIS MATERIAL, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

Trademarks

Synopsys and certain Synopsys product names are trademarks of Synopsys, as set forth at <https://www.synopsys.com/company/legal/trademarks-brands.html>

All other product or company names may be trademarks of their respective owners.

Free and Open-Source Software Licensing Notices

If applicable, Free and Open-Source Software (FOSS) licensing notices are available in the product installation.

Third-Party Links

Any links to third-party websites included in this document are for your convenience only. Synopsys does not endorse and is not responsible for such websites and their practices, including privacy practices, availability, and content.

Synopsys, Inc.
www.synopsys.com

Software Sample Terms and Conditions

These Software Sample Terms and Conditions ("Terms") cover the Software you license from Synopsys, unless and until we enter into new terms that expressly replace these Terms. If you use the Software as an employee of or for the benefit of your company, your company will be the licensee under these Terms. Accepting it you consent to these Terms on behalf of yourself and the company on whose behalf you will use the Software. The effective date of these Terms is the date that you click accepted them. If you do not agree to these Terms or if you do not have the power and authority to accept these Terms on behalf of your company, you may not use the Software and Synopsys is unwilling to provide you with them.

1. The Software is not an item of Licensed Software or Licensed Product under any end-user software license agreement with Synopsys or any supplement thereto. Synopsys hereby grants to you a limited, personal, non-exclusive, non-transferable, non-assignable, fully paid, royalty free, worldwide, perpetual license to use the Software, and create modifications of the components of the Software provided to you in source code format, solely for use with a Synopsys DesignWare IP. All modifications of the Software are owned by Synopsys, and you hereby irrevocably assign ownership of those modifications (and all intellectual property rights therein) to Synopsys. However, you are under no obligation to disclose any such modifications to Synopsys and the modifications are automatically licensed to you as Software. The Software and all modifications are the Confidential Information of Synopsys, and you agree not to distribute or disclose them.

2. THE SOFTWARE IS PROVIDED BY SYNOPSYS SOLELY ON AN "AS IS" BASIS AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE HEREBY DISCLAIMED. IN NO EVENT SHALL SYNOPSYS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THE SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

3. These Terms, which can be modified only by Synopsys in writing, shall be governed by and construed under the laws of the State of California, USA, without regard for its conflict of laws principles. You may not transfer or assign your license rights to any other person in any manner (by assignment, operation of law or otherwise) unless you have obtained written consent from Synopsys. If you attempt to transfer or assign any of your license rights without Synopsys's consent, the transfer or assignment will be ineffective, null, and void. For purposes of this Section, a transfer or assignment of your license rights will be deemed to have occurred (a) if a third party (or group of third parties acting in concert) acquires beneficial ownership of fifty percent (50%) or more of either your assets or of the stock or other equity interests entitled to vote for your directors or equivalent managing authority, or (b) in the event of a merger, consolidation or other business combination between you and one or more third parties where your stockholders immediately before that transaction own (directly or indirectly), after that transaction, less than fifty percent (50%) of the stock or other equity interests entitled to vote for the directors or equivalent managing authority of the surviving entity. These Terms constitute the entire understanding and agreement between you and Synopsys with respect to the subject matter hereof and supersedes and replaces all prior and contemporaneous understandings and agreements, oral or written, express or implied, regarding the same subject matter.

Contents

Chapter 1	
MIPI MSHC CTest Overview	7
1.1 CTest Overview	8
1.2 CTest Features	8
1.3 CTest MSHC Drive Features	9
1.4 Deliverables	9
Chapter 2	
Software Package	11
2.1 Directory Structure	12
2.2 Software Package Content	13
Chapter 3	
Architecture	15
3.1 CTest Architecture	16
3.2 Platform or Hardware Abstraction Layer	17
3.3 MHSC Bare Metal driver Layer	17
3.4 MMC/SD Core Driver Layer	19
3.5 Command Execution Engine Layer (CEE)	22
3.6 SD Card Layer (CEE)	23
3.7 Application Layer	25
3.8 I/O Request Sequence Flow	26
3.9 Test Execution Engine (TEE)	28
3.10 CTest Common Software Components	29
3.10.1 Message Queues (CMsgQueue)	29
3.10.2 CTask (CTest Task scheduler)	30
3.10.3 CWatch Component	30
3.10.4 CLog (CTest Logging Feature)	31
3.10.5 CTest Utility/Tools	32
3.10.6 Parser	32
3.11 Terminal Framework	32
Chapter 4	
Build Instruction	35
4.1 Tools Used for Building and Running the MSHC CTest	36
4.2 Building the CTest Software Package	36
4.3 MSHC Application Usage.	36
Chapter 5	
CTest Porting Guide	39

5.1 Porting Platform/HAL Abstraction	40
5.2	MMC Host Stack 41
5.3 Use of FSM Logic in no-os Platform.	41
Chapter 6	
MSHC Driver Known Limitation	45
Appendix A	
Input Configuration File Format	47
A.1 Input Test configuration file format	47
Appendix B	
Sample Configuration File	49

1

MIPI MSHC CTest Overview

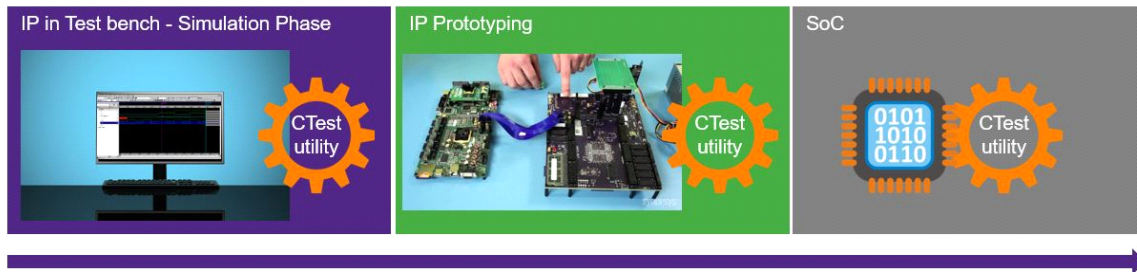
This chapter provides a high level description of the CTest Software sample, It includes CTest software components, architecture details

- [“CTest Overview”](#) on page 8
- [“CTest Features”](#) on page 8
- [“CTest MSHC Drive Features”](#) on page 9
- [“Deliverables”](#) on page 9

1.1 CTest Overview

CTest is software package which help to accelerate customer's SoC bring-up and IP validation. With flexible framework support and Bare-Metal driver, the Software package supports to execute CTests on many platforms and validate designs using diverse kind of tests like conformance, functional, robustness tests etc.

Figure 1-1 CTests - From Simulation Phase to SoC



CTests SW utility enables consistent bring up and validation along the project phase in different platforms. It can be used all way down from initial Test Bench to SoC.

CTests can be used for comparing the functional behavior in different platforms; Also, it can be used in recreating the issues identified in HW platforms in the simulation for better debugging.

CTests provide huge scope for the configurability i.e. controlling the test cases (enable/disable, repeat counts), environment (instances enabled / disabled).

The CTests framework provides software infrastructure to integrate the DWC IP bare metal device driver for hardware validation over NO-OS (without operating system) based environment.

The CTests includes other supporting software components common to facilitate implementation of layered driver architecture and other tools such as terminal software for interactive debugging & test automation.

CTest Framework can be used to implement the bare metal drivers for DW IPs like MSHC (Mobile storage Host controller), DWC-I3C, DWC-XGMAC etc.

1.2 CTest Features

CTest Framework enables the user to develop the driver for no-OS platform, CTest framework includes various re-usable software components as mentioned below

- CTask - For creating task for no-OS platform, it is OS independent abstraction.
- CMessage Queues - for intertask communication of messages.
- CLink - Communication link API between two task/application through shared memory.
- CLog - Debug Logging facility.
- CWatch - Useful to watch variables through terminal commands.
- Terminal - Utility over stdio or UART for user interaction with CTest and driver software to easy debugging and testing.
- Parser - To read driver or test configuration input files.
- TEE - Test Execution engine to run automation test scripts and logging the results.

1.3 CTest MSHC Drive Features

The MSHC driver features are as follows:

- Support for basic SD commands for MMC SD/card enumeration
- Support for basic SD commands for eMMC card enumeration
- Support for speed mode, Default speed, High speed
- Support for SDR12 speed mode
- Support for SDR25 speed mode
- Support for SDR50 speed mode
- Support for SDR104 speed mode
- Support for HS200 speed mode (eMMC)
- Support for 4-bit bus width
- Support for 8-bit bus width
- Support for data transfer commands, read single/multiple block
- Support for data transfer commands, write single/multiple block
- Support for SD Tuning block

1.4 Deliverables

The release under this project will include the following content as specified in [Table 1-1](#)

Table 1-1 Deliverables

Deliverable	Format	Description
Bare metal driver for Synopsys DesignWare® MIPI I3C controller.	C source & header files	I3C controller driver source code
Synopsys CTest software.	C source & header files	CTest software source code
Documentation – User Guide	Word file	User Guide for CTest I3C.

2

Software Package

This chapter describes the Synopsys CTest Software package, It includes following topics:

- [“Directory Structure”](#) on page 12
- [“Software Package Content”](#) on page 13

2.1 Directory Structure

Figure 2-1 Directory Structure



2.2 Software Package Content

Table 2-1 list the software package and their content.

Table 2-1 Software Package Content

Software Folder/File Name	Description
C_tests/build	All output files (.o) will be created in this directory
C_tests/core	Core CTest reusable components
C_tests/drv	Portable Bare-Metal driver source code
C_tests/inc	Common header files
C_tests/platfrom	Hardware abstraction layer and Platform support code for PC, ARC board and Simulators (SV-DPI)
C_tests/os	OS abstraction layer
C_tests/test	CTest Test Execution Engine
C_tests/example	CTest components Examples source code
C_tests/utlis	CTest Utility like parser, terminal etc
C_tests/main	CTest Main application code
C_tests/tools	CTest Additional tools or application

3

Architecture

This chapter describes the Synopsys MSHC CTest Software Architecture and Block diagram. The topics includes the following:

- [“CTest Architecture” on page 16](#)
- [“Platform or Hardware Abstraction Layer” on page 17](#)
- [“MHSC Bare Metal driver Layer” on page 17](#)
- [“MMC/SD Core Driver Layer” on page 19](#)
- [“Command Execution Engine Layer \(CEE\)” on page 22](#)
- [“SD Card Layer \(CEE\)” on page 23](#)
- [“Application Layer” on page 25](#)
- [“I/O Request Sequence Flow” on page 26](#)
- [“Test Execution Engine \(TEE\)” on page 28](#)
- [“CTest Common Software Components” on page 29](#)
- [“Terminal Framework” on page 32](#)

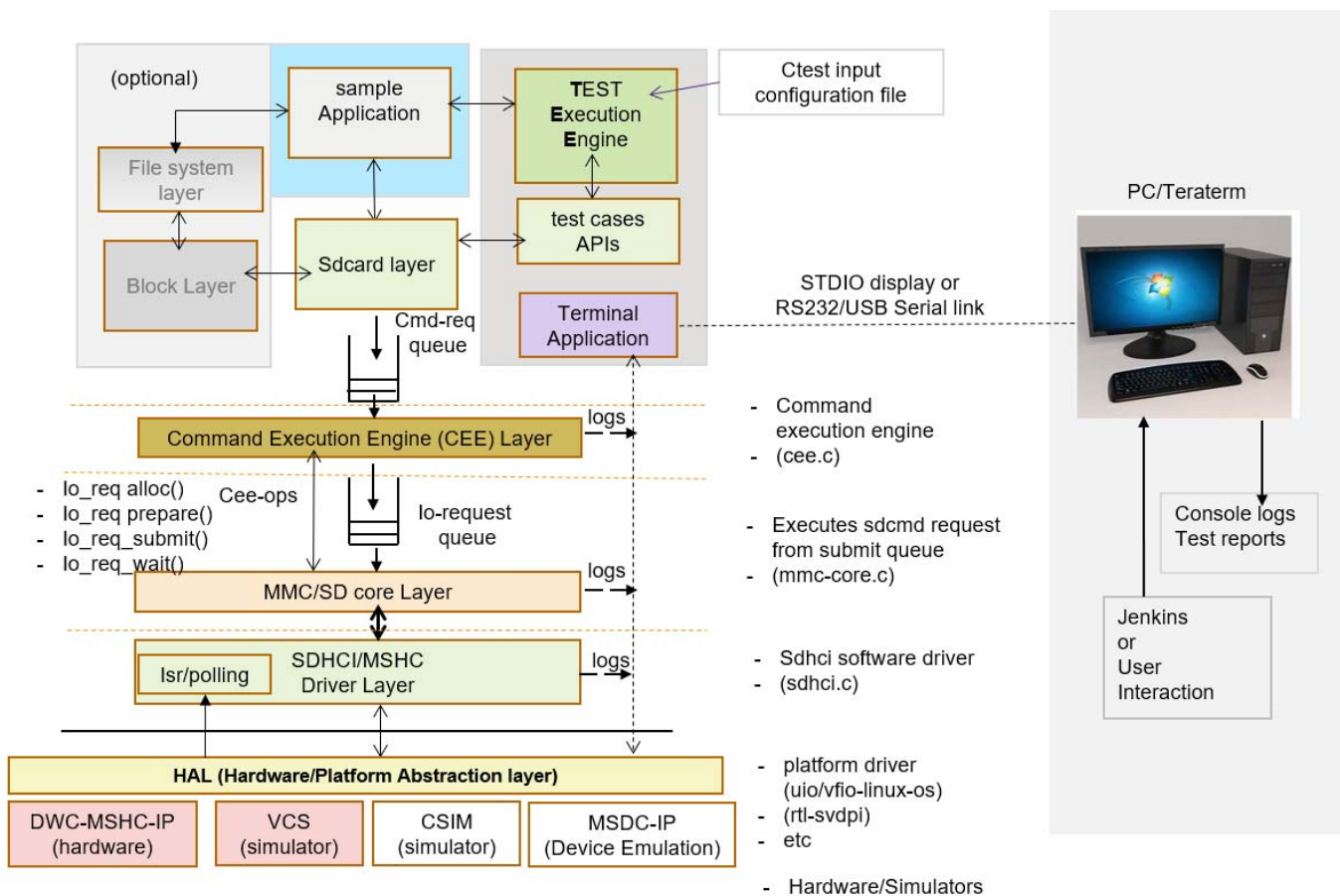
3.1 CTest Architecture

The figure-1 shows the block diagram of C-Test based MSHC driver architecture. It is designed based on layered approach, consists of

1. Platform/Hardware abstraction layer
2. Mass Storage Host Controller (MSHC) driver layer
3. MMC/SD core driver layer (manages io_request submission/completion).
4. Command Execution Engine layer (CEE) - handles commands submitted by SD card layer.
5. SD Card layer (handles sdcmd execution, like device enumeration, configuring of device to different speed & modes, performs basic read/write blocks)

C-Tests suite includes reusable software components and other tools, like terminal client/server for user interactive debugging and testing.

Figure 3-1 MSHC Architecture Block Diagram



Each layer is an independent software component which communicates with other layer through well defined interface through interface-ops (interface operations/functions) or through message queues.

3.2 Platform or Hardware Abstraction Layer

The hardware/platform abstraction layer is a platform dependent layer provides an access to device resources, which includes the following:

1. Register read/write access to device.
2. Interrupt handling (Not supported yet).
3. DMA mapping (Not supported yet)

The HAL layer is ported on the custom platform on which C-Tests MSHC driver is running, if Linux platform is used, then HAL layer is ported on Linux platform, in case of Linux platform, Linux PCI support, uio/vfio kernel modules support are used. If RTL simulation platform is used, the HAL layer is ported to RTL SVDPI interface to access MSHC controller. Similarly, HAL layer can be ported to any custom platform.

Table 3-1 HAL Interface APIs

No	API	Description
1	uint8_t hal_read8(struct hal_obj_t *hal, void *addr)	Reads 8-bit data from specified register address
2	uint16_t hal_read16(struct hal_obj_t *hal, void *addr);	Reads 16-bit data from specified register address
3	uint32_t hal_read(struct hal_obj_t *hal, void *addr);	Reads 32-bit data from specified register address
4	uint8_t hal_write8(struct hal_obj_t *hal, void *addr, uint8_t val);	Writes 8-bit data to specified register address
5	uint16_t hal_write16(struct hal_obj_t *hal, void *addr, uint16_t val);	Writes 16-bit data to specified register address
6	uint32_t hal_write(struct hal_obj_t *hal, void *addr, uint32_t val);	Writes 32-bit data to specified register address
7	uint32_t hal_reg<8/16/32>_set_field(struct hal_obj_t *hal, void *addr, uint8_t st_bit, uint8_t nbits, uint8_t val);	Set the value in specified fields of register address
8	struct hal_obj_t *hal_register(uint32_t (*read)(void *pdata, void *addr, uint8_t grain), uint32_t (*write)(void *pdata, void *addr, uint32_t val, uint8_t grain), void *p_data)	Register to hal layer by providing the read/write access method.

3.3 MSHC Bare Metal driver Layer

The mass storage host controller Driver is low level driver based on SDHCI specification provides support to configure the mshc/sdhci host controller. The sdhci driver access the devices through registered HAL interface functions and provides services to mmc_core core layer through mmc_host_ops as shown below.

```
struct mmc_host_ops_t {
    /* send mmc commands */
    int (*send_cmd)(struct sdhci_t *sdhci, struct io_req_t *req);
    int (*io_ctrl)(struct sdhci_t *sdhci, uint32_t cmd, uint32_t args);
}
```

```

int (*get_card_detect)(struct mmc_dev_t *mmc_dev);
int (*get_card_write_protect)(struct mmc_dev_t *mmc_dev);
int (*start_io_tuning)(struct mmc_dev_t *mmc_dev, struct io_req_t *req);
int (*enable_clock)(struct sdhci_t *sdhci, uint8_t speed_mode,
                    uint8_t mmc_vdd, uint32_t mmcclk_clock);
int (*disable_clock)(struct sdhci_t *sdhci);
int (*set_speed)(struct sdhci_t *sdhci, uint8_t speed_mode);
int (*set_data_width)(struct sdhci_t *sdhci, uint8_t width);
int (*set_voltage)(struct sdhci_t *sdhci, uint8_t mmc_vdd_sel, uint8_t
mmc_vdd_volt);
}

```

Table 3-2 MSHC/SDHCI Driver APIs

No	sdhci Low Level Driver API	Description
1	void sdhci_reset(struct sdhci_t *sdhci, uint8_t mask)	Reset the sdhci controller
2	void sdhci_enable_intr(struct sdhci_t *sdhci)	Enable the sdhci interrupts
3	void sdhci_disable_intr(struct sdhci_t *sdhci)	Disable the sdhci interrupts
4	void sdhci_io_ctrl(struct sdhci_t *sdhci, uint32_t cmd, uint32_t args)	Perform sdhci-io controls like enable/disable power, etc.
5	int sdhci_setup_clock(struct sdhci_t *sdhci, uint32_t clock)	Enable the sdhci clock
6	int sdhci_disable_clock(struct sdhci_t *sdhci)	Disable the sdhci clock
7	struct sdhci_t *sdhci_register(addr_t base_addr, uint8_t irq)	Register the sdhci host controller
8	int sdhci_set_data_width(struct sdhci_t *sdhci, uint8_t width)	Set data width
9	int sdhci_send_cmd(struct sdhci_t *sdhci, struct io_req_t *io_req)	Configure specified sd-cmd based on io-request
10	int sdhci_wait_for_cmd_complete(struct sdhci_t *sdhci)	Wait for command completion
11	void sdhci_complete_request(struct sdhci_t *sdhci)	Complete the io-request.
12	int sdhci_handle_cmd_intr(struct sdhci_t *sdhci, uint32_t intr_mask)	Handle the command interrupts
13	int sdhci_handle_data_intr(struct sdhci_t *sdhci, int32_t intr_mask)	Handle the data interrupts
14	int sdhci_irq_handler(uint8_t task_id, void *data)	Handles the sdhci interrupt events
15	int sdhci_init(struct sdhci_t *sdhci)	Initialize the sdhci controller
16	int sdhci_module_init(void)	Initialize sdhci module
17	struct sdhci_t *sdhci_alloc(void)	Allocate sdhci instance

3.4 MMC/SD Core Driver Layer

The mmc/sd core layer interfaces between sdhci driver and command execution engine (CEE) through *struct cee_ops* interface functions, mm core driver handles the io_request allocation, io_request preparation, io_request submission and io_request completion.

The mmc_core init function registers the

```
struct cee_ops_t {
    void *priv_data;
    int (*submit_request)(void *priv_data, struct io_req_t *io_req);
    int (*prepare_io_req)(void *priv_data, struct io_req_t *io_req, void *param,
uint32_t len);
    int (*release_io_req)(void *priv_data, struct io_req_t *io_req);
    int (*get_io_status)(void *priv_data, struct io_req_t *io_req);
};
```

Table 3-3 MMC/SD Driver APIs

No	API	Description
1	int mmc_core_submit_request (struct mmc_dev_t *mmc_dev, struct io_req_t *cmd_req);	Submit a io_request to mmcore io_request queue
2	void mmc_core_request_complete (struct io_req_t *cmd_req);	Does the completion of io-request.
3	int mmc_core_init (struct mmc_dev_t *mmc_dev, struct mmc_host_t *host);	Initializes the mmc_core module
4	int mmc_core_request_handler (uint8_t task_id, void *args)	Handle the io_request submitted to io_request queue.
5	int mmc_clock_enable (struct mmc_dev_t *mmc_dev, uint8_t enable_1p8v, uint32_t delay_ms)	Enables the mmc clock.
6	int mmc_clock_disable (struct mmc_dev_t *mmc_dev)	Disable the mmc clock
7	int mmc_set_bus_width (struct mmc_dev_t *mmc_dev, uint8_t bus_width)	Set mmc data bus width.
8	struct mmc_req_t *mmc_request_alloc(uint8_t flags)	Allocate mmc io_request
9	void mmc_request_free (struct mmc_req_t *req)	Free the mmc io_request
10	struct mmc_dev_t *mmc_dev_alloc (uint8_t flags)	Allocate mmc device context.
11	int prepare_mmc_cmd0_go_idle (struct mmc_dev_t *mmc_dev, struct io_req_t *io_req, struct cmd_param_s *cmd_param, uint32_t param_len)	Prepare the io_request form cmd0 based on input parameters.

Table 3-3 MMC/SD Driver APIs

No	API	Description
12	int prepare_mmc_cmd3_send_rel_addr (struct mmc_dev_t *mmc_dev, struct io_req_t *io_req, struct cmd_param_s *cmd_param, uint32_t param_len)	Prepare the io_request form cmd3 based on input parameters
13	int prepare_mmc_cmd11_voltage_switch (struct mmc_dev_t *mmc_dev, struct io_req_t *io_req, struct cmd_param_s *cmd_param, uint32_t param_len)	Prepare the io_request form cmd11 based on input parameters
14	int prepare_mmc_cmd8_send_if_cond (struct mmc_dev_t *mmc_dev, struct io_req_t *io_req, struct cmd_param_s *cmd_param, uint32_t param_len)	Prepare the io_request form cmd8 based on input parameters
15	int prepare_mmc_cmd2_send_cid (struct mmc_dev_t *mmc_dev, struct io_req_t *io_req, struct cmd_param_s *cmd_param, uint32_t param_len)	Prepare the io_request form cmd2 based on input parameters
16	int prepare_mmc_cmd7_card_select (struct mmc_dev_t *mmc_dev, struct io_req_t *io_req, struct cmd_param_s *cmd_param, uint32_t param_len)	Prepare the io_request form cmd7 based on input parameters
17	int prepare_mmc_cmd9_send_csd (struct mmc_dev_t *mmc_dev, struct io_req_t *io_req, struct cmd_param_s *cmd_param, uint32_t param_len)	Prepare the io_request form cmd9 based on input parameters
18	int prepare_mmc_acmd41_init_card (struct mmc_dev_t *mmc_dev, struct io_req_t *io_req, struct cmd_param_s *cmd_param, uint32_t param_len)	Prepare the io_request form acmd41 based on input parameters
19	int prepare_mmc_acmd51_send_scr (struct mmc_dev_t *mmc_dev, struct io_req_t *io_req, struct cmd_param_s *cmd_param, uint32_t param_len)	Prepare the io_request form cmd51 based on input parameters
20	int prepare_mmc_cmd13_send_status (struct mmc_dev_t *mmc_dev, struct io_req_t *io_req, struct cmd_param_s *cmd_param, uint32_t param_len)	Prepare the io_request form cmd13 based on input parameters
21	int prepare_mmc_cmd6_send_status (struct mmc_dev_t *mmc_dev, struct io_req_t *io_req, struct cmd_param_s *cmd_param, uint32_t param_len)	Prepare the io_request form cmd6based on input parameters
22	int prepare_mmc_cmd17_read_one_block (struct mmc_dev_t *mmc_dev, struct io_req_t *io_req, struct cmd_param_s *cmd_param, uint32_t param_len)	Prepare the io_request form cmd17 based on input parameters
23	int prepare_mmc_cmd18_read_multi_block(struct mmc_dev_t *mmc_dev, struct io_req_t *io_req, struct cmd_param_s *cmd_param, uint32_t param_len)	Prepare the io_request form cmd18 based on input parameters

Table 3-3 MMC/SD Driver APIs

No	API	Description
24	int prepare_mmc_cmd23_set_blockcnt (struct mmc_dev_t *mmc_dev, struct io_req_t *io_req, struct cmd_param_s *cmd_param, uint32_t param_len)	Prepare the io_request form cmd23 based on input parameters
25	int prepare_mmc_cmd24_write_one_block (struct mmc_dev_t *mmc_dev, struct io_req_t *io_req, struct cmd_param_s *cmd_param, uint32_t param_len)	Prepare the io_request form cmd24 based on input parameters
26	int prepare_mmc_cmd55_nxt_is_acmd (struct mmc_dev_t *mmc_dev, struct io_req_t *io_req, struct cmd_param_s *cmd_param, uint32_t param_len)	Prepare the io_request form cmd55 based on input parameters
27	int mmc_submit_request (void *priv_data, struct io_req_t *io_req)	Submit the request to io_request queue.
28	int mmc_prepare_io_req (void *priv_data, struct io_req_t *io_req, void *param, uint32_t len)	Prepare the mmc io_request
29	int mmc_release_io_req (void *priv_data, struct io_req_t *io_req)	releases the mmc io_request
30	int mmc_get_io_status (void *priv_data, struct io_req_t *io_req)	Get the mmc io_request status
31	void mmc_cmd_init (void)	mmc command initialization
32	void mmc_dev_init (struct mmc_dev_t *mmc_dev)	initialize device core context.
33	struct mmc_dev_t *mmc_dev_new (uint32_t flags)	Allocate new mmc device core context.

3.5 Command Execution Engine Layer (CEE)

The Command Execution Engine Layer (CEE) is an generic layer which interfaces between driver core layer and application layer. CEE handles the commands submitted to cmdreq queue by allocating the io_request, preparing the io_request and submitting and handles the completion of io_request. CEE handles the io_request using cee interface functions cee_ops_t as shown below.

```
struct cee_ops_t {
    void *priv_data;
    int (*submit_request)(void *priv_data, struct io_req_t *io_req);
    int (*prepare_io_req)(void *priv_data, struct io_req_t *io_req, void *param,
uint32_t len);
    int (*release_io_req)(void *priv_data, struct io_req_t *io_req);
    int (*get_io_status)(void *priv_data, struct io_req_t *io_req);
};
```

Table 3-4 Command Execution Engine APIs

No	API	Description
1	uint8_t cee_submit_request(struct cee_t *cee, struct io_req_t *io_req)	CEE submit io_request to mmc_core layer.
2	uint8_t cee_prepare_io_req(struct cee_t *cee, struct io_req_t *io_req, void *param, uint32_t len)	CEE prepares io_request
3	uint8_t cee_release_io_req(struct cee_t *cee, struct io_req_t *io_req)	CEE release the io_request
4	uint8_t cee_get_io_status(struct cee_t *cee, struct io_req_t *io_req)	CEE get the status of submitted io_request.
5	uint8_t cee_init(uint8_t level)	CEE module initialization.
6	struct cee_t *cee_register(char *name, struct cee_ops_t *ops)	Registering to CEE layer.
7	int cee_execute_cmd(struct cee_t *cee, struct io_req_t *io_req)	Executing the io_request command.
8	void io_req_init(uint8_t task_id, struct io_req_t *io_req)	Initialize the io_request.
9	int cee_core_task(uint8_t task_id, void *args)	CEE core task which handles the command submitted to CEE request queue.

3.6 SD Card Layer (CEE)

The SD-Card layer implements SD commands and handles the SD Card device enumeration as per SDHCI specification as shown in [Figure 3-2](#).

Figure 3-2 MMC/SD card Device Enumeration Sequence

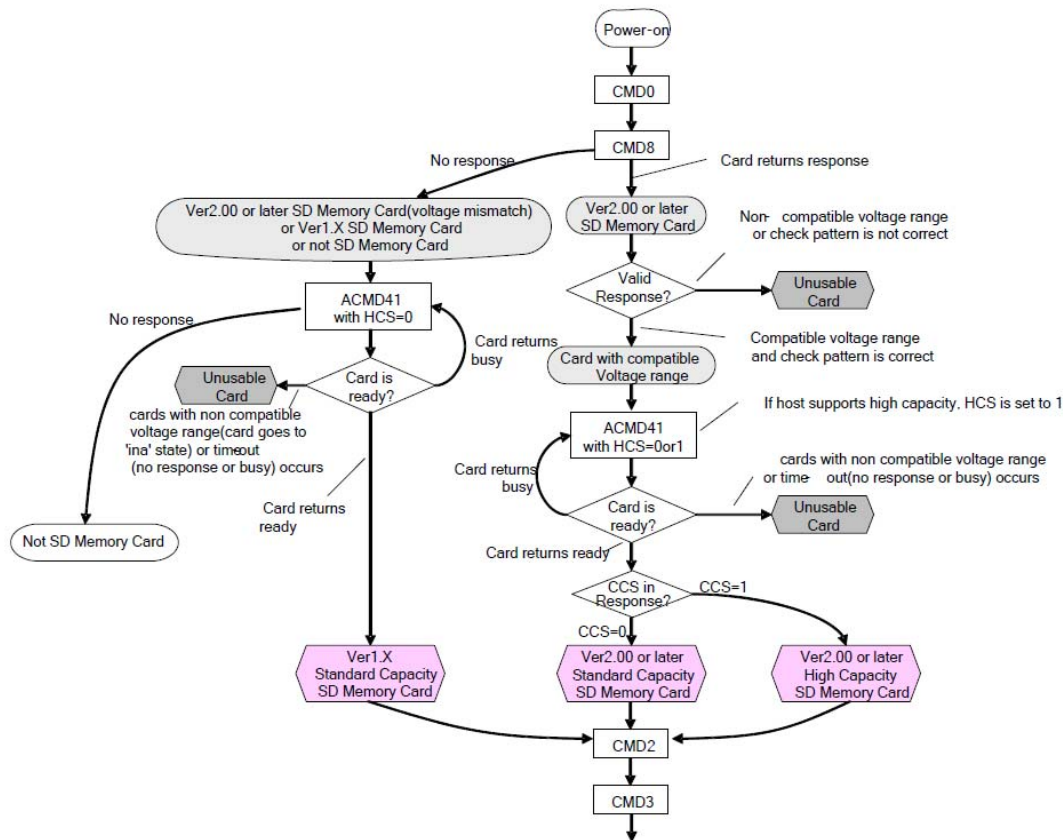


Table 3-5 list the supported SD-CMDS, the SD Card layer uses the below commands for SD card device enumeration as per sequence shown in Figure 3-2.

Table 3-5 SD-CMDS APIs

No	API	Description
1	void mmc_cmds_init(uint8_t flags);	Initialize the mmc cmds module
2	int mmc_cmd0_card_set_idle(struct mmc_dev_t *mmc_dev, struct cmd_param_t *mmc_cmds, uint8_t boot_mode);	Execute mmc/sd cmd0, First command issued card, set the card to idle state.
3	int mmc_cmd3_send_rel_adr(struct mmc_dev_t *mmc_dev, struct cmd_param_t *mmc_cmds, uint8_t rca);	Host request card to publish a new relative card address.

Table 3-5 SD-CMDS APIs

No	API	Description
4	int mmc_cmd11_voltage_switch(struct mmc_dev_t *mmc_dev, struct cmd_param_t *mmc_cmds, uint32_t args);	Executes mmc/sd cmd11, Host issue cmd11 to invoke voltage switch sequence based on if SR18A =1 in response of acmd41.
5	int mmc_cmd2_get_card_cid(struct mmc_dev_t *mmc_dev, struct cmd_param_t *mmc_cmds, uint32_t args);	Executes mmc/sd cmd2, host gets the cid of mmc/sd card.
6	int mmc_cmd7_card_select(struct mmc_dev_t *mmc_dev, struct cmd_param_t *mmc_cmds, uint16_t rca);	Executes mmc/sd cmd7, host select one card and put it into transfer state.
7	int mmc_cmd9_get_csd(struct mmc_dev_t *mmc_dev, struct cmd_param_t *mmc_cmds, uint16_t rca);	Executes mmc/sd cmd9, host get card specific data structure (csd).
8	int mmc_cmd13_send_status(struct mmc_dev_t *mmc_dev, struct cmd_param_t *mmc_cmds, void *card_status_buf, uint32_t buflen);	Executes mmc/sd cmd13 gets the mmc/sd card status.
9	int mmc_acmd13_send_status(struct mmc_dev_t *mmc_dev, struct cmd_param_t *mmc_cmds, void *card_status_buf, uint32_t buflen);	Executes mmc/sd acmd13, host issues this command to get the sd-status of the card (512bits).
10	int mmc_cmd6_switch_function(struct mmc_dev_t *mmc_dev, struct cmd_param_t *mmc_cmds, uint8_t speed_mode, void *databuf, uint32_t buflen);	Executes mmc/sd cmd6, host issue this command to select speed mode.
11	int mmc_acmd6_switch_function(struct mmc_dev_t *mmc_dev, struct cmd_param_t *mmc_cmds, uint8_t bus_width);	Executes mmc/sd acmd6 , host issues this command to set the sd bus data width.
12	int mmc_acmd6_set_buswidth(struct mmc_dev_t *mmc_dev, struct cmd_param_t *mmc_cmds, uint8_t bus_width);	Executes mmc/sd acmd6 set bus width by configuring input parameters and returns success or failure.
13	int mmc_read_block(struct mmc_dev_t *mmc_dev, struct cmd_param_t *mmc_cmds, uint32_t block_addr, uint32_t block_cnt, uint8_t *databuf, uint32_t buflen);	Executes mmc/sd cmd17 or cmd18, host issues this command to read single/multiple block.
14	int mmc_set_block_cnt(struct mmc_dev_t *mmc_dev, struct cmd_param_t *mmc_cmds, uint32_t block_cnt);	Executes mmc/sd cmd23 to set the block count.
15	int mmc_write_block(struct mmc_dev_t *mmc_dev, struct cmd_param_t *mmc_cmds, uint32_t block_addr, uint32_t block_cnt, uint8_t *databuf, uint32_t buflen);	Executes mmc/sd cmd24 or cmd25, host issues this command to read single/multiple block.
16	int mmc_cmd8_check_card_intf_condition(struct mmc_dev_t *mmc_dev, struct cmd_param_t *mmc_cmds, uint8_t volt_supplied, uint8_t check_pattern);	Executes mmc/sd cmd8, host issues this command to check interface operating condition.

Table 3-5 SD-CMDS APIs

No	API	Description
17	int mmc_cmd55_next_is_acmd(struct mmc_dev_t *mmc_dev, struct cmd_param_t *mmc_cmds, uint16_t rca);	Executes mmc/sd cmd55 , host issue this command to specify the card, that next command followed this is application specific command (ACMDx).
18	int mmc_acmd41_check_card_op_condition(struct mmc_dev_t *mmc_dev, struct cmd_param_t *mmc_cmds, uint8_t is_sdxc_support, uint8_t use_1p8v, uint8_t is_max_perf, uint8_t is_ocr);	Executes mmc/sd acmd41, host issues this command to set the host capability, current signal voltage, and power saving or max performance of the card.

3.7 Application Layer

The main application function, perform the initialization of C-Tests modules, driver core layer modules and performance the mmc/sd device enumeration, further user can perform read/write to mmc/sd card. Refer to sample main application source file.

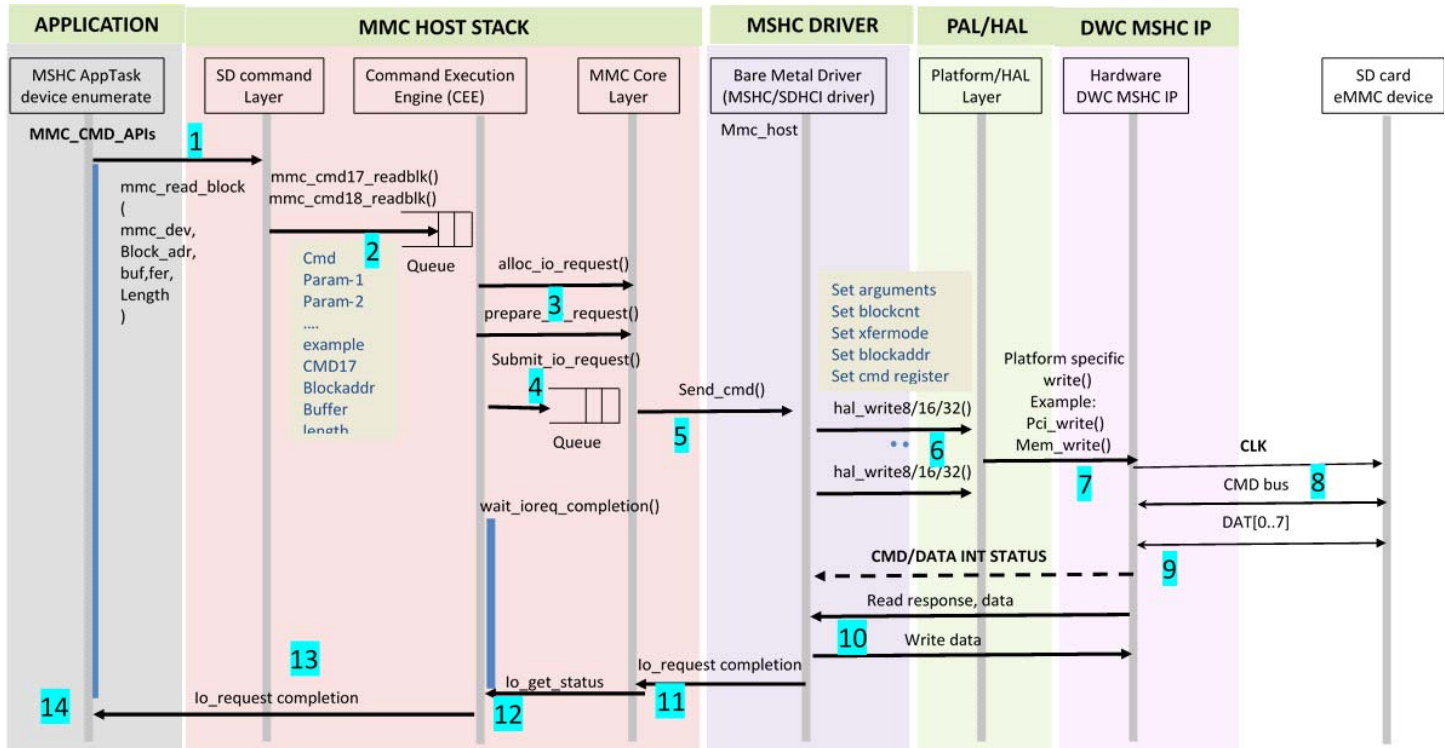
Table 3-6 mmc/sd APIs

No	API	Description
1	void app_main(int level)	Performs the initialization of C-Test core and driver core modules.
2	int mmc_app_task(uint8_t task_id, void *args)	Performs mmc/sd card device enumeration if card is present, perform intended user defined custom functionalities.
3	int main(int argc, char *argv[])	Main entry function.

3.8 I/O Request Sequence Flow

The Figure 3-3 shows the sequence diagram of mmc/sd command i/o request flow. The sequence number labeled 1 to 14 details the flow of i/o request from application layer to device and completion of i/o request.

Figure 3-3 MMC/SD Command I/O Request Sequence Flow



1. The application task or device enumerate function, issues the sequence of commands (as per enumeration sequence in figure-2) by calling SD_CMD APIs, (refer to SD command layer section). For example, the `mmc_read_block()` invokes by application to read blocks by passing parameters, sd-device context, block address, number of blocks, buffer and buffer length.
2. The SD_CMD layer API, `mmc_read_block()` forms one or more `mmc_cmd` request and submit the request to CEE input queue one at a time and wait for command completion.
3. The MMC_CORE layer reads the CEE input queue and allocates the `io_request` and prepares the I/O request.
4. The `io_request` is submitted to `io_request` input queue.
5. The mmc core request handler task, reads the `io_request` and configures the MSHC host controller by `mmc-host` interface function `sendcmd()` API.
6. The `sendcmd` API configures the MSHC host controller by sequence writes (`hal_write8/16/32()` API), write to argument register, set the block count, block size, set block address and finally initiates the command writing to `cmd_to` host controller command register.
7. The `hal_write8/16/32()` API invokes registered platform specific API for device access for read/write operation.

8. The MSHC/SDHCI host controller sends the command to device on CMD line, and gets the response and DATA (if any) on DAT lines for read/write transfers.
9. The MSHC controller generates command/data/response events.
10. The MSHC/SDHCI driver interrupt service routine or polling function reads the INTR STATUS register and handles the corresponding interrupts.
11. Once the io_request is completed, the polling thread or ISR function invokes callback routine and does the completion of io_request.
12. CEE layer waiting for command completion by polling for io_request status and does the completion of MMC command request submitted by SD_CMD layer.
13. SD Command layer completes the request and handles the next command request if required.
14. The Application task or device enumeration function waiting for completion of request, gets completed based on success or failure analyze the received response and data.

3.9 Test Execution Engine (TEE)

The CTest engine used for test automation, which sequences and executes the test cases based on user input configuration and logs the result.

The test execution engine module, reads the test input configuration file, sequences and executes each test-cases provided in the input configuration file.

The driver can register to TEE framework providing the list of TEE interface functions as follows.

```
struct tee_ops_t {
    void *priv_data;
    struct cmd_param_t *(*alloc_cmd_req)(struct test_case_t *tc);
    int (*prepare_test_case)(struct test_case_t *tc);
    int (*run_test_case)(struct test_case_t *tc);
    int (*get_status)(struct test_case_t *tc);
    int (*release_test_case)(struct test_case_t *tc);
    int (*update_tc_param)(struct test_case_t *tc, struct key_val_t *param);
};
```

Table 3-7 TEE APIs

No	API	Description
1	struct tee_t *alloc_tee(char *name);	Allocate an instance of TEE object.
2	int tee_init(void);	Initialize the TEE module.
3	struct tee_t *register_tee(char *name, struct tee_ops_t *ops, char *tc_cfg_file);	Register to TEE.
4	int tee_execute_test_case(struct test_case_t *tc);	Execute the specific test-case by invoking the test-case function and logs the result.
5	int tee_core_task(uint8_t task_id, void *args);	TEE core task, reads the test configuration input file and build all the test-case, then sequence and execute each test case based on the sequence control.
6	int start_tee(struct tee_t *tee);	Start the TEE engine.
7	int stop_tee(struct tee_t *tee);	Stop the TEE engine
8	int read_config_data(char *inp_str, struct str_map_t *str_map, uint8_t len, uint8_t type, void *data_val);	Helper function to extract the key-value configuration data.
9	int tee_load_testcases(struct tee_t *tee, char *tc_cfg_file);	Reads the input test configuration file and builds the test-case database.

3.10 CTest Common Software Components

3.10.1 Message Queues (CMsgQueue)

The CTest message queue is a reusable component referred to as *CMsgQueue* is a circular queue implemented over shared memory or non-shared memory with static configurable number of queues and dynamic allocation of depth (in bytes) for each queue.

The shared/non-shared queue memory is partitioned into two sections, queue management or control section and queue data section. The queue control section includes allocated queue control information like queue number, queue name, offset to data section, queue read/write pointers. The queue data section includes the queue data.

In case of no-OS embedded platform (without operating system), shared or non-shared queued memory is same memory allocated either statically or dynamically from RAM. For user space application on Linux-OS, the CTest OS abstraction layer provides API to allocate the shared memory.

Table 3-8 List of Message Queue APIs

No	API	Description
1	int cmsg_q_module_init(uint32_t flags);	called once during application init.
2	int cmsg_q_create(int q_size, char *name);	Creates a named queue with depth of size in bytes
3	int cmsg_q_send(uint8_t msg_q_num, void *buffer, uint16_t len);	Send a message (buffer) of length (len) in bytes to specified msg_q_num.
4	int cmsg_q_receive(uint8_t msg_q_num, void *buffer, uint16_t len);	Receive a message (buffer) at most len (length in bytes) from queue.
5	int cmsg_q_is_empty(uint8_t msg_q_num);	Check whether the queue is empty.
6	int get_cmsg_q_num(char *name);	Get queue number from queue-name
7	int sm_cmsg_q_module_init(uint32_t flags, void *sm_qmem_addr, uint32_t qmem_size);	called once during application init.
8	int sm_cmsg_q_create(int q_size, char *name);	Creates a shared memory queue with depth of size in bytes
9	int sm_cmsg_q_send(uint8_t msg_q_num, void *buffer, uint16_t len);	Send a message (buffer) of length (len) in bytes to shared memory queue.
10	int sm_cmsg_q_receive(uint8_t msg_q_num, void *buffer, uint16_t len);	Receive a message (buffer) at most len (length in bytes) from shared memory queue.
11	int sm_cmsg_q_is_empty(uint8_t msg_q_num);	Check whether the queue is empty.
12	int get_sm_cmsg_q_num(char *name);	Get queue number from queue-name.

3.10.2 CTask (CTest Task scheduler)

In no-os environment, there are no way to create the task/thread, however there exist only two threads, one main application thread running in background (in no-interrupt context) and other interrupt service routines executed during interrupts from device (interrupt context). The main application thread in background runs in while loop which invokes each function/task sequentially. The CTask provides an OS abstraction for no-os platform to create/suspend/resume/sleep a task and schedules these tasks at background.

This facilitates the user to develop the application software or driver in modular or layer architecture and message queues are used for inter-task communication. The [Table 3-9](#) list the CTask abstraction API, the CTask scheduler in os-less platform is basically a function scheduler invoked in main application program in while loop, which can be controlled through task_suspend/resume/sleep APIs. CTask abstraction API can be ported any OS by using *ctask_os_register()* API.

Table 3-9 List of CTask APIs

No	API	Description
1	int ctask_create(char *name, int (*fn)(uint8_t task_id, void *task_pdata), void *task_pdata, uint32_t arg2, uint32_t arg3, uint32_t arg4);	Creates a task with specified name, fn is pointer to function to be invoked when this task gets called.
2	int ctask_enable(uint8_t task_id);	Enables and add task to schedule list.
3	int ctask_suspend(uint8_t task_id);	Suspend the task.
4	int ctask_resume(uint8_t task_id);	Resume the task.
5	int ctask_sleep(uint8_t task_id, uint32_t delay_ms);	Sleep or delay task for specified amount of time in millisecond.
6	int ctask_exit(uint8_t task_id);	Free the task from scheduler list.
7	void ctask_scheduler(void);	CTask scheduler function which invokes all ready task in sequential order.
8	int ctask_get_fnstate(uint8_t task_id);	Get the state of specified function attached to task_id.
9	int ctask_set_fnstate(uint8_t task_id, uint8_t nxt_state);	Save/Modify the state of specified function attached to task_id.
10	int ctask_os_register(struct ctask_os_t *os);	Register a OS abstraction API to CTask.

3.10.3 CWatch Component

The CWatch is an software component used to watch/view or modify a specified variable or memory location. Generally this is used in terminal to watch the user defined variable, or to watch important data variables of any software component like CTask to check list of task and its state, list of message queues and its read/write pointers of CMsgQueue component, etc. This is generic framework used by any software component used to watch/modify the data of interest. Refer [Table 3-10](#) for list of CWatch APIs.

Table 3-10 List of CWatch APIs

No	API	Description
1	int cwatch_init(void);	Initialize cwatch module, called once during application init.
2	int add_to_cwatch(char *name, void *addr, uint8_t type);	Adding a variable to watch by passing its name, address and data type to cwatch. The data-type would be 0 to 7. 0 – unsigned 8 bit data (XUINT8) 1 – unsigned 16 bit data (XUINT16) 2 – unsigned 32 bit value (XUINT32) 3 – signed 8 bit data (XINT8) 4 – signed 16 bit data (XINT16) 5 – signed 32 bit data (XINT32) 6 – string data (XSTRING) 7 – floating point data (XFLOAT)
3	int del_from_cwatch(char *name);	deleting a variable entry from cwatch.
4	void print_cwatch_data(struct cwatch_t *watch);	Print all variables added to cwatch.
5	void print_cwatch(char *str);	Print specified variable by its name.

3.10.4 CLog (CTest Logging Feature)

Clog provides the logging of debug information at multiple levels (0 to 9) for verbosity. The *clog_print()* API used to log the message with specified logging level. The [Table 3-11](#) list the clog API and the description.

Table 3-11 List of CLog APIs

No	API	Description
1	void set_clog_level(uint8_t level);	Set current log level logs all messages below or equal set current level.
2	void set_clog_output(uint8_t output);	Print the log to specified output, the result output is as follows: 0 - CLOG_STDIO – standard i/o 1 – CLOG_TERMINAL – print to terminal 2 – CLOG_FILE – log to file
3	uint8_t clog_print(uint8_t level, char *fmt);	Clog printf to log the message with user defined log level.

Table 3-12 List of cLog Levels

Log level	Value	Description
0	CLOG_NONE	Disable logging

Table 3-12 List of cLog Levels

Log level	Value	Description
1	CLOG_ERR	To log the errors
2	CLOG_INFO	To log important message or information
3	CLOG_WARN	To log warning messages
4 - 9	CLOG_LEVEL_4 to CLOG_LEVEL_9	User defined logging levels

3.10.5 CTest Utility/Tools

The CTest includes utilities and tools useful for developer for debugging, reading configuration files etc.

Table 3-13 CTest Utility/tools

No	Utility	Description
1	Parser	The parser used to read the key value pair (<key> = <value>) from input configuration file.
2	Terminal	The terminal framework used to develop terminal utility application over message queues or communication link like UART, etc.

3.10.6 Parser

The parser is utility used to read the key value pair (<key> = <value>) from input file like configuration or INI file.

Table 3-14 List of Parser API

No	API	Description
1	int read_config_file(char *config_file)	Read the specified configuration file and creates a dictionary of value pair.
2	int get_config_data(char *inp_str, char *out_str, uint8_t out_str_len);	Returns the value string associate with given input key value string.
3	void print_dictionary(int level);	Print the value pairs read from configuration file with specified log level.

3.11 Terminal Framework

The Terminal is an debug console over communication link like UART or through shared memory queue, useful for developer for interactive debugging, like register read/write, memory dump or run specific test case or to run test automation.

The terminal framework consists of terminal core registered with *terminal_ops_t* terminal operation, which waits for user input character (getch() API) over registered communication link and accepts the command

(new line terminator), and execute the specified command prints the result to output terminal console using `putch()` API.

```
struct terminal_ops_t {  
    int (*init)(void *priv_obj);  
    int (*getch)(void *priv_obj, char *ch);  
    int (*putch)(void *priv_obj, char ch);  
    int (*uninit)(void *priv_obj);  
};
```

Table 3-15 List of Terminal API

No	API	Description
1	<code>int terminal_core_init(void);</code>	Initialize terminal core, called once during initialization
2	<code>int terminal_create(void *priv_obj, struct terminal_ops_t *ops, char *prompt);</code>	Create a terminal with specified terminal operations over UART or message queues with any custom prompt string ("CTEST >").
3	<code>int terminal_register_cmd(char *cmdstr, void (*cmd_func)(void *terminal, token_t *token, int max_token));</code>	Add a custom command, can be invoked from terminal console.
4	<code>int terminal_puts(struct terminal_t *t, char *str);</code>	Prints the string on specified terminal.

4

Build Instruction

The section provides information on how to build and run the MSHC CTest software package. This includes the following topics:

- [“Tools Used for Building and Running the MSHC CTest”](#) on page 36
- [“Building the CTest Software Package”](#) on page 36
- [“MSHC Application Usage.”](#) on page 37

4.1 Tools Used for Building and Running the MSHC CTest

Table 4-1 describes the tools used to build and run the MSHC CTest software package

Table 4-1 Tools Used

Element	Version	Purpose
DWC-MSHC Controller IP	1.80-ea04, 1.80a	BIT FILE
FPGA Board	HAPSDX7	FPGA platform
GNU Tool Chain	4.8.5	GNU Tool chain
DeskTop PC	Ubuntu 14.04	To build MHSC CTest and testing mmc/sd card or eMMC device, the device access is through PCIe based HAPSDX7 FPGA platform hosting DWC-MSHC-IP connected to PC.
FPGA BOARD	HAPSDX7	HAPSDX7 FPGA board hosting DWC MSHC IP ver1.80 connected to PC through PCIe interface
CTest MHSC package	1.0	CTest software sample for MHSC driver
SD Card	Sony 32GB SD UHS II	SONY SD Card

4.2 Building the CTest Software Package

1. The top level CTest directory includes Makefile, which compiles and builds the MHSC CTest application using the configuration file for MSHC driver is available in configuration directory.
 - Config/config.h - Top level CTest configuration file
 - Config/mshc_config.h - MSHC driver configuration
2. GNU tool chain version 4.8.5 used for build CTest.
3. Change to C_Test directory


```
# cd c_tests
```
4. Make clean


```
# Make clean
```
5. Building MHSC CTest


```
#make
```

The object files (.o) and mshc_main executable application will be created in C_tests/build directory.

4.3 MSHC Application Usage.

The MSHC application command line arguments are as follows

```
# ./build/mshc_main [options]
```

The cmdline options to test SD card are as follows

```
#./build/mshc_main -d 6 -speed_mode 3 -xfer_mode 0 -bus_width 1 -mmcm_clk 100 -emmc 0 -
stop_at_cmd 1
```

The cmdline options to test eMMC Device are as follows

```
#./build/mshc_main -d 6 -speed_mode 2 -xfer_mode 0 -bus_width 1 -mmcm_clk 100 -emmc 1 -
stop_at_cmd 1
```

Table 4-2 Option

Options	Descriptions
-t <input_test_cfg_file>	The list of commands or tests to be executed can be provided in input test_configuration file. Please refer to format of input_test_cfg_file in Appendix A, "Input Configuration File Format"
-d <debug_level>	The debug_level value to 3 to 9 for verbosity driver logs.
--speed_mode <value>	The speed mode <value> is as follows ■ 0-SDR12 ■ 1-SDR25 ■ 2-SDR50 ■ 3-SDR104 ■ 4-DDR50 The eMMC speed mode <value> is as follows ■ 0-eMMC default speed ■ 1-eMMC High speed ■ 2-eMMC HS200 ■ 3-eMMC HS400 (Not supported)
--xfer_mode <0/1/2>	Driver mode, by default <1> PIO mode is chosen ■ 0- PIO (supported) ■ 1 – SDMA (Not supported) ■ 2 – ADMA2 (Not supported)
--bus_width <1/2>	Bus width selection either 4 or 8 bit mode ■ <1>- 4bit buswidth ■ <2>- 8bit buswidth
--vdd <1>	Use 1.8V
--mmc_clk <clk-in-Mhz>	■ mmcm-clock frequency in Mhz ■ Supported values are 100/125 Mhz, 200Mhz for eMMC

Table 4-2 **Option**

Options	Descriptions
--tx_phase <value>	Tx_phase value from 0 to 127
--rx_phase <value>	Rx_phase value from 0 to 127
--emmc <1>	Select for emmc device
--stop_at_cmd <1>	Stop and ask user input before proceed to execute next command
--l <log-file>	Provide log file name

5

CTest Porting Guide

CTest porting guide provides details how to how to port the MSHC-CTest for your hardware or embedded board. This includes the following topics:

- [“Porting Platform/HAL Abstraction”](#) on page 40
- [“MMC Host Stack”](#) on page 41
- [“Use of FSM Logic in no-os Platform.”](#) on page 41

5.1 Porting Platform/HAL Abstraction

The bare metal driver needs to access device (read/write) through Platform abstraction/HAL abstraction APIs, refer to `hal_register()` API how to register the platform specific read/write methods. In order to execute CTest application on Linux platform or Ubuntu machine, CTest uses the platform/HAL abstraction which uses Linux “UIO driver” (user space i/o driver) as platform/HAL abstraction for device register read/write access

The MHSC/SDHCI driver sequence of porting as follows

The sdhci device structure

```
struct sdhci_t {
...
struct hal_obj_t *hal; /* pointer to hal object */
struct hal_obj_t *pal; /* pointer to pal object */
struct uio_pci_t *uio; /* pointer to uio object */
...
struct mmc_host_t host;
struct clk_mod_t *mmcm_clk;
}
```

1. The bare metal sdhci driver implemented in source file `c_tests/drv/mmc/dwc-mshc/sdhci.c`. The sdhci host controller structure `struct sdhci_t` includes the pointer to hal, pal and uio platform abstraction object. The `sdhci_init()` sample pseudo code below shows sequence of step to initialize the hal/pal/uio objects.

```
#define SNPS_VENDOR_ID0x16c3
#define SNPS_DEVICE_ID0xc202 //for dwc mshc ip
Struct sdhci_t *sdhci_0;
int sdhci_init()
{
    /* allocate the sdhci instance */
    Sdhci_0 = sdhci_alloc();
    if (sdhci == NULL)
        return ERROR_OPER_FAIL;

    /* do uio_pci_init, which scan pci device for given vendor id and device-id gets the mmap
    device
                                address to access device */
    sdhci->uio = uio_pci_init(SNPS_VENDOR_ID, SNPS_DEVICE_ID, NULL, NULL);
    if (sdhci->uio == NULL)
        return ERROR_OPER_FAIL;
    /* register sdhci device access read/write method to hal layer */
    sdhci->hal = hal_register(sdhci_read, sdhci_write, sdhci);
    if (sdhci->hal == NULL)
        return ERROR_OPER_FAIL;

    /* register platform specific sdhci read/write, which does read/write uio mmap address */
    sdhci->pal = hal_register(sdhci_uio_read, sdhci_uio_write, sdhci);
    if (sdhci->pal == NULL)
        return ERROR_OPER_FAIL;
    ...
    /* further initialization follows */

    return 0;
}
```

2. The `sdhci_init()` function mentioned above, allocates one instances of `sdhci` controller object and initializes the `uio` driver, which gets the mmap of device pci base address for mshc register read/write access.
3. The `sdhci_read()/sdhci_write()` is an wrapper API, which inturn invokes the platfrom specific read/write functions `sdhci_iomem_read()/sdhci_iomem_write()`.

```
uint32_t sdhci_uio_read(void *pdata, void *addr, uint8_t grain)
{
    reg_offs = ((addr_t)addr - sdhci->io_base);
    if (grain == 1)
        return *(volatile uint8_t *) (sdhci->uio->map.io_mem + reg_offs);
    else if (grain == 2)
        return *(volatile uint16_t *) (sdhci->uio->map.io_mem + reg_offs);
    else if (grain == 4)
        return *(volatile uint32_t *) (sdhci->uio->map.io_mem + reg_offs);
    else
        return -1;
}
```

The `sdhci_uio_write()` API would b similar to above `sdhci_uio_read()`.

4. Refer to `C_tests/drv/mmc/dwc-mshc/sdhci_hal.c` for detailed implementation.

5.2 MMC Host Stack

The MMC host stack (refer figure in section 3.8) includes one or more layer implemented above bare metal driver MSHC driver. The mmc host stack handles io request submitted by application and uses the `sdhci` host ops (struct `mmc_host_ops_t` refer section) to configure the controller and issue SD commands. The MMC host stack wait for io request completion and return the status to the application layer.

Refer to respective section for details about MMC host stack layers.

5.3 Use of FSM Logic in no-os Platform.

The CTest based application is designed to execute on no-os (without operating system) platform, implementation of layered architecture driver stack would be challenging, as each layer consists of one or more task which provides specfic services (generally io request handling) to layer above and layer below till all through application layer. Hence preserving state of each task is important. Hence the CTest components and driver stack uses FSM logic (Finite State machine) to remember current state and makes the transition to next state based on specific functionality implementation. Each state is local to the particular task or function.

One such example is Command Execution Engine (CEE) which takes the request from input request queue and prepare the `io_request` and submit the request to core layer below and waits till completion of the request.

The sample pseudo code of CEE core task as shown below.

```
int cee_core_task(uint8_t task_id, void *args)
{
#define XSTATE0_CEE_INIT          0
#define XSTATE1_CEE_PREP          1
#define XSTATE2_CEE_EXEC          2
#define XSTATE3_CEE_WAITIO        3
```

```

    struct cee_t *cee;
    int retval, i, len;
    int state = ctask_get_fnstate(task_id);
    int nxt_state = state;
    cee = (struct cee_t *)args;
    if (cee == NULL)
        return 0;
    switch(state) {
    case XSTATE0_CEE_INIT: //Wait till any request queued by application
        if (cmsg_q_is_empty(cee->inp_req_queue))
            return 1;
        //extract the request
        cee->cmd_param = NULL;
        len = cmsg_q_receive(cee->inp_req_queue, &cee->cmd_param,
sizeof(addr_t));
        if (cee->cmd_param == NULL)
            return 1;
        clog_print(CLOG_LEVEL3, "%s rcvd cmdno(%d), param_len = %d\n", __func__,
        cee->cmd_param->cmd, len);
        nxt_state = XSTATE1_CEE_PREP;
    case XSTATE1_CEE_PREP: //prepare the request by calling driver specific prepare
interface API
        retval = cee_prepare_io_req(cee, &cee->io_req, cee->cmd_param,
        sizeof(struct cmd_param_s));
        if (retval != 0) {
            clog_print(CLOG_ERR, "%s: failed to prepare cmd-req\n", cee-
>name);
            return 1;
        }
        io_req_init(task_id, &cee->io_req);
        cee->io_req.resp = &cee->cmd_param->resp[0];
        nxt_state = XSTATE2_CEE_EXEC;
        clog_print(CLOG_LEVEL3, "submitting cee io_request\n");
    case XSTATE2_CEE_EXEC:
        // execute the command
        retval = cee_execute_cmd(cee, &cee->io_req);
        if (retval < 0) {
            clog_print(CLOG_LEVEL5, "unable to execute cmd (error=%d)\n",
retval);
            return 1;
        }
        nxt_state = XSTATE3_CEE_WAITIO;
        break;
    case XSTATE3_CEE_WAITIO: // wait for IO completion
        if (cee->io_req.status != IO_STATUS_IN_PROGRESS) {
            cee->cmd_param->status = cee->io_req.status;
            cee->io_stat.tot_cnt++;
            if (cee->io_req.status == IO_STATUS_SUCCESS) {
                clog_print(CLOG_LEVEL5, "io_req is sucess\n");
                cee->io_stat.pass_cnt++;
                for (i = 0; i < 4; ++i)
                    clog_print(CLOG_LEVEL6, "<==resp[%d] = %x\n",
                    i, cee->io_req.resp[i]);
            }
        }
    }
}

```

```
        } else {
            clog_print(CLOG_ERR, "io_req failed with error:%d\n",
                      cee->io_req.status);
            cee->io_stat.fail_cnt++;
        }
        // release the io request
        cee_release_io_req(cee, &cee->io_req);
        //re-initialize the io-request to INIT state for next execution
        io_req_init(task_id, &cee->io_req);
        nxt_state = XSTATE0_CEE_INIT;
    }
    break;
default:
    break;
}
cee_core_task_state = nxt_state;
ctask_set_fnstate(task_id, nxt_state);
return 1; // return 1 indicates this task never exits and runs forever by the scheduler.
}
```


6

MSHC Driver Known Limitation

Following MSHC Driver Features not supported

- Interrupt mode not supported.
- DMA mode (SDMA/ ADMA2/ ADMA3) is not supported.
- HS400 speed mode is not supported.
- CQE feature not supported.

Input Configuration File Format

A.1 Input Test configuration file format

The CTest application can take input test configuration file, where user can provide list of test commands to be executed sequentially. The TEE (Test Execution Engine) reads this input configuration file and executes the list of commands.

Each line of test input configuration consists in the format of key value pairs (<key>=<value>), which can be test command, test parameters, and few control commands to control the sequence of execution.

Table A-1 Input Configuration Syntax

Key Value Pair	Description
# <comment string>	Comment line will be ignored
TC_SEQ_NUM = <value>	The test sequence number used to control the sequence of commands to be executed if required.
TC_DESC = <test case description>	Testcase description string. Example: TC_DESC = "MMC-card device enumeration"
TC_ID = TC_MSHC_MMC_DEV_ENUMERATE	ID for Test case database
TC_PARAM0<N>_<cmd-specific-paramer> = <value>	List of parameters for command. Example For MMC_card device enumeration, Following are the list of parameters - TC_PARAM01_SPEED_MODE = 3 - TC_PARAM02_BUS_WIDTH = 1 - TC_PARAM03_XFER_MODE = 0 - TC_PARAM04_EMMC_VDD = 1 - TC_PARAM05_MMCM_CLOCK s= 100000000 - TC_PARAM06_USR_INTF = 1

Table A-1 Input Configuration Syntax

Key Value Pair	Description
TC_EXEC = <y/n>	Whether to execute this test-case or not. ■ Y - execute this test case ■ N - skip this test case
TC_EXEC_NEXT = <TC_SEQ_NUM>	Its is optional, indicates the next test case to be executed in sequence. ■ 0 - Ignore and follow the next test-case ■ <1..N> - execute the specific test case.

B

Sample Configuration File

Example B-1 shows the sample "mmc_sd_cfg.txt".

Example B-1 Sample "mmc_sd_cfg.txt"

```
# MMC Test cases
TC_SEQ_NUM = 1
TC_DESC = "MMC-card device enumeration"
TC_ID = TC_MSHC_MMC_DEV_ENUMERATE
TC_PARAM01_SPEED_MODE = 3
TC_PARAM02_BUS_WIDTH = 1
TC_PARAM03_XFER_MODE = 0
TC_PARAM04_EMMC_VDD = 1
TC_PARAM05_MMCM_CLOCK = 100000000
TC_PARAM06_USR_INTF = 1
TC_EXEC = y
TC_EXEC_NXT = 0

# MMC read block Test cases
TC_SEQ_NUM = 2
TC_DESC = "MMC-card write block"
TC_ID = TC_MSHC_MMC_WRITE_BLOCK
TC_PARAM01_BLOCK_ADR = 0x50100
TC_PARAM02_BLOCK_CNT = 5
TC_PARAM03_DATA_USER_PAT = 0xBB
TC_PARAM04_USE_RAND_PAT = 1
TC_EXEC = y
TC_EXEC_NXT = 0

# MMC read block Test cases
TC_SEQ_NUM = 3
TC_DESC = "MMC-card read block"
TC_ID = TC_MSHC_MMC_READ_BLOCK
TC_PARAM01_BLOCK_ADR = 0x0
TC_PARAM02_BLOCK_CNT = 2
TC_EXEC = y
TC_EXEC_NXT = 0
```

```
# MMC read block Test cases
TC_SEQ_NUM = 4
TC_DESC = "MMC-card read block"
TC_ID = TC_MSHC_MMC_READ_BLOCK
TC_PARAM01_BLOCK_ADR = 0x50100
TC_PARAM02_BLOCK_CNT = 5
TC_EXEC = y
TC_EXEC_NXT = 0
```